INTERNET OF THINGS (IoT)

UNIT 5 — Protocols and IoT Cloud

Complete Exam-Ready Study Notes
Prepared from: unit-5.pptx
Prepared for: Tanish Uchhadiya

Covered topics: IoT Cloud • UART • Arduino Serial • SoftwareSerial • Cloud Computing • IaaS/PaaS/SaaS • Fog Computing • Blynk IoT Platform • IoT Application Architecture • Exam Practice

These notes explain every major topic from the PPT in simple English. They include definitions, explanations, diagrams, tables, code explanations, exam questions, MCQs and revision material.

Table of Contents

Chapter	Topic	Page / Section
1	IoT Application Building Using IoT Cloud	3
2	UART Serial Communication	3
3	Arduino Serial Class and Functions	4
4	Reading and Writing Serial Data	5
5	Communication Between Two Arduino Boards	6
6	SoftwareSerial Library	7
7	Cloud Computing	8
8	Cloud Service Models: IaaS, PaaS and SaaS	9
9	Fog Computing	11
10	IoT Cloud and Blynk	13
11	Blynk Components and Virtual Pins	14
12	Blynk NodeMCU Code	15
13	Common IoT Application Architecture	16
14	Quick Revision and Exam Preparation	18

Chapter 1: IoT Application Building Using IoT Cloud

This unit explains how IoT applications are built using communication protocols and IoT cloud platforms.

1.1 Simple Definition

IoT Cloud is an internet-based platform that helps IoT devices connect, send data, store data, analyse data and be controlled remotely.

1.2 Topics Covered in This Unit

- IoT cloud and IoT cloud configuration.
- Creating an account on an IoT cloud platform.
- Common architecture for building IoT applications.
- Introduction to communication protocols.
- UART serial communication.
- Software Serial.
- IoT applications.
- Fog computing and cloud computing.
- Blynk IoT cloud platform.

1.3 Why Do IoT Applications Need a Cloud?

- Sensors collect data from the real world.
- The controller sends this data to the cloud through Wi-Fi, mobile data or Ethernet.
- The cloud stores and analyses the data.
- The user can see the data through a mobile app or web dashboard.
- The cloud can send alerts or commands back to actuators.

> **Example:** In a smart home, a temperature sensor sends readings to the cloud. If temperature becomes too high, the cloud or controller can switch on a fan automatically.

Chapter 2: UART Serial Communication

2.1 Simple Definition

UART means **Universal Asynchronous Receiver-Transmitter**. It is a serial communication protocol used to send data between an Arduino board and another device.

2.2 Detailed Explanation

In serial communication, data is sent one bit at a time through a communication line. UART is called **asynchronous** because it does not use a separate clock signal. Instead, both devices agree in advance on the same communication speed, called the **baud rate**.

2.3 Key Terms

Term	Simple meaning
Serial communication	Data is sent one bit after another.
Asynchronous	No common clock wire is used; both devices use the same baud rate.
Baud rate	Speed of communication in bits per second (bps), such as 9600 or 115200.
TX	Transmit pin; sends data.
RX	Receive pin; receives data.
Data bits	Bits in each data unit; Arduino commonly uses 8 data bits.
Buffer	Temporary memory where incoming serial data is stored before reading.

2.4 Important Characteristics

- UART is simple, low-cost and easy to implement.
- It is one of the most commonly used device-to-device serial protocols.
- Data format and transmission speed are configurable.
- It uses TX and RX pins.
- Both connected devices must use the same baud rate.

2.5 UART Connection Diagram

Arduino A Arduino B
TX (Pin 1) ■■■■■■■■■■■■ RX (Pin 0)
RX (Pin 0) ■■■■■■■■■■■■ TX (Pin 1)
GND ■■■■■■■■■■■■■■■■■■■■ GND

2.6 Important Rule

TX of one device must connect to RX of the other device. TX-to-TX and RX-to-RX connections will not work correctly.

2.7 Arduino UNO UART Pins

Board	UART Pins	Important note
Arduino UNO	Pin 0 = RX0, Pin 1 = TX0	These pins are also used for USB communication with Arduino IDE.
Arduino UNO	Only one hardware UART	Disconnect wires from pins 0 and 1 before uploading a sketch.
Other Arduino boards	May have additional UART ports	Check the board specification.

2.8 Common Mistakes

- Using different baud rates on sender and receiver.
- Connecting TX to TX instead of TX to RX.
- Leaving wires connected to pins 0 and 1 while uploading code.
- Forgetting to connect GND between two boards.

Chapter 3: Arduino Serial Class and Functions

3.1 Simple Definition

The Arduino **Serial class** is used to communicate through UART. It allows Arduino to read and write data through the serial port.

3.2 Main Serial Functions

Function	Purpose	Example
Serial.begin(rate)	Starts serial communication at a selected baud rate.	Serial.begin(9600);
Serial.end()	Ends serial communication.	Serial.end();
Serial.available()	Returns the number of bytes available for reading.	if (Serial.available() > 0)
Serial.read()	Reads one byte from the serial buffer.	char c = Serial.read();
Serial.peek()	Reads one byte without removing it from the buffer.	int x = Serial.peek();
Serial.flush()	Waits for outgoing serial data transmission to complete.	Serial.flush();
Serial.write()	Sends raw byte data.	Serial.write(65);
Serial.print()	Sends text or value without a new line.	Serial.print("Hi");
Serial.println()	Sends text or value and adds a new line.	Serial.println("Hi");

3.3 Function Groups

Group	Functions
Start / stop communication	Serial.begin(), Serial.end()
Check serial port	if(Serial), Serial.available(), Serial.availableForWrite()
Read data	Serial.read(), Serial.peek(), Serial.readBytes(), Serial.readBytesUntil(), Serial.readString(), Serial.readStringUntil()
Write / print data	Serial.write(), Serial.print(), Serial.println()
Manage transmission	Serial.flush()

3.4 print() vs println()

Function	Output on Serial Monitor	New line?
Serial.print("Hello")	Hello	No
Serial.println("Hello")	Hello	Yes

3.5 read() vs peek()

Function	Reads data?	Removes data from buffer?
Serial.read()	Yes	Yes
Serial.peek()	Yes	No

Chapter 4: Reading and Writing Serial Data

4.1 Reading Data From Serial Port

The following program waits until data arrives from the Serial Monitor, reads it and displays it.

```
int incomingByte = 0;

void setup() {
Serial.begin(9600);
}

void loop() {
if (Serial.available() > 0) {
incomingByte = Serial.read();
Serial.print("I received: ");
Serial.println(incomingByte, DEC);
}
}
```

4.2 Line-by-Line Explanation

Code line	Meaning
int incomingByte = 0;	Creates an integer variable to store incoming serial data.
void setup()	This function runs once when Arduino starts.
Serial.begin(9600);	Starts serial communication at 9600 bits per second.
void loop()	This function runs again and again.
if (Serial.available() > 0)	Checks whether at least one byte has arrived.
incomingByte = Serial.read();	Reads one byte and stores it in incomingByte.
Serial.print("I received: ");	Prints text without moving to a new line.
Serial.println(incomingByte, DEC);	Prints the received byte in decimal form and moves to a new line.

4.3 Writing Data Using Serial.write()

Serial.write() sends raw data through the serial port.

Syntax	Meaning
Serial.write(data);	Sends one byte or value.
Serial.write(buf, len);	Sends an array named buf containing len bytes.

```
void setup() {
Serial.begin(9600);
}

void loop() {
Serial.write('A');
Serial.write(0x04);
delay(1000);
}
```

4.4 Explanation

- Serial.begin(9600) starts communication at 9600 baud.
- Serial.write('A') sends the byte value of character A.
- Serial.write(0x04) sends hexadecimal value 04.
- delay(1000) waits for one second before repeating.

4.5 Exam Point

> **Serial.print()** is mainly used to display readable text or numbers. **Serial.write()** is mainly used to send raw byte data.

Chapter 5: Communication Between Two Arduino Boards

5.1 Concept

Two Arduino boards can communicate using UART. One board acts as a **sender** and the other acts as a **receiver**.

5.2 Connection

Sender Arduino Receiver Arduino
TX (Pin 1) ■■■■■■■■■■■■■■■■ RX (Pin 0)
RX (Pin 0) ■■■■■■■■■■■■■■■■ TX (Pin 1)
GND ■■■■■■■■■■■■■■■■■■■■■■■■ GND

5.3 Sender Code

```
char mystr[5] = "Hello";

void setup() {
Serial.begin(9600);
}

void loop() {
Serial.write(mystr, 5);
delay(1000);
}
```

5.4 Sender Code Explanation

- char mystr[5] = "Hello"; creates a character array containing Hello.
- Serial.begin(9600) starts serial communication at 9600 baud.
- Serial.write(mystr, 5) sends five bytes from the array.
- delay(1000) sends the message once every second.

5.5 Receiver Code

```
char mystr[10];

void setup() {
Serial.begin(9600);
}

void loop() {
Serial.readBytes(mystr, 5);
Serial.println(mystr);
delay(1000);
}
```

5.6 Receiver Code Explanation

- char mystr[10]; creates an array to store received data.
- Serial.begin(9600) uses the same baud rate as the sender.
- Serial.readBytes(mystr, 5) reads five bytes into the array.
- Serial.println(mystr) displays the received message on the Serial Monitor.
- delay(1000) repeats the process after one second.

5.7 Important Points

- Both boards must use the same baud rate.
- The sender sends five characters: H, e, l, l, o.
- The receiver reads exactly five bytes.
- A common ground connection is required.

Chapter 6: SoftwareSerial Library

6.1 Simple Definition

SoftwareSerial is an Arduino library that creates an additional serial port using normal digital pins instead of the fixed hardware UART pins.

6.2 Why Is SoftwareSerial Needed?

Arduino UNO has only one hardware serial port on pins 0 and 1. If these pins are already used for USB communication or another device, SoftwareSerial can create another serial connection on other digital pins.

6.3 Hardware UART vs SoftwareSerial

Feature	Hardware UART	SoftwareSerial
Pins used	Fixed RX/TX pins	Any selected digital pins
Performance	Faster and more reliable	Slower because software handles communication
Simultaneous TX and RX	Usually supported	Cannot transmit and receive at the same time
Multiple ports	Limited by hardware	Possible, but only one can receive at a time
Best use	Main serial communication	Extra low-speed serial device

6.4 Limitations of SoftwareSerial

- It cannot transmit and receive data at the same time.
- If multiple SoftwareSerial ports are used, only one can receive data at a time.
- It is less efficient and slower than hardware UART.

6.5 Syntax

```
#include
SoftwareSerial mySerial(RX_PIN, TX_PIN);
```

6.6 Example Code

```
#include

#define RX_PIN 10
#define TX_PIN 11

SoftwareSerial mySerial(RX_PIN, TX_PIN);

void setup() {
mySerial.begin(9600);
Serial.begin(9600);
Serial.println("SoftwareSerial Ready");
}

void loop() {
if (Serial.available()) {
char data = Serial.read();
mySerial.write(data);
Serial.print("Sent to SoftwareSerial: ");
Serial.println(data);
}

if (mySerial.available()) {
char data = mySerial.read();
Serial.print("Received from SoftwareSerial: ");
Serial.println(data);
}
}
```

6.7 Code Explanation

- #include adds the SoftwareSerial library.
- RX_PIN 10 means pin 10 will receive data.

- TX_PIN 11 means pin 11 will send data.
- SoftwareSerial mySerial(10, 11) creates a new serial object.
- mySerial.begin(9600) starts the software serial port.
- Serial.begin(9600) starts the hardware serial port.
- The loop sends data received from hardware serial to software serial.
- It also reads data from software serial and prints it on the hardware serial monitor.

Chapter 7: Cloud Computing

7.1 Simple Definition

Cloud computing is the delivery of computing services such as servers, storage, databases, networking, software and analytics over the internet.

7.2 Formal Definition From PPT

> Cloud computing is a model for enabling universal, convenient, on-demand network access to a shared pool of configurable computing resources, such as networks, servers, storage, applications and services, which can be rapidly provided and released with minimal management effort.

7.3 Easy Explanation

In cloud computing, the application and data are stored on remote servers instead of only on the user computer. The user accesses these services through the internet.

> **Traditional computing:** Application runs on the user computer.
> **Cloud computing:** Application runs in the cloud and the user accesses it through the internet.

7.4 Benefits of Cloud Computing

- Provides large storage and processing capability at low cost.
- Resources can be provided quickly when required.
- Users pay only for the resources they use.
- Reduces the need to buy and maintain physical servers.
- Reduces operating costs for servers, storage, databases, networking, software and analytics.
- Allows services to be accessed from anywhere through the internet.
- Large companies such as Amazon, Google and Facebook use cloud services to deliver services over the internet.

7.5 Real-Life Example

> **Google Drive:** Your files are stored on remote servers. You can open them from a phone or computer using the internet. You do not need to manage the physical storage device.

Chapter 8: Cloud Service Models

There are three main cloud service models: **IaaS**, **PaaS** and **SaaS**. Each model provides a different level of service and control.

8.1 Infrastructure as a Service (IaaS)

IaaS provides raw computing resources such as processing power, storage and networking in a virtual data centre.

- The user can install an operating system.
- The user can deploy and run applications.
- The user manages the operating system, storage and applications.
- The cloud provider manages the physical hardware and underlying infrastructure.
- It gives the highest level of control among the three models.

8.2 Platform as a Service (PaaS)

PaaS provides a development environment for application developers.

- It provides computing and storage infrastructure.
- It also provides a development platform layer.
- It may include web servers, databases, SDKs, APIs, libraries and development tools.
- The developer builds, manages and maintains the application.
- The provider manages the platform and infrastructure.

8.3 Software as a Service (SaaS)

SaaS provides a complete software application to the user. The service provider runs and manages the application.

- The user does not need to manage software updates or server maintenance.
- It is commonly used through a subscription, such as monthly or yearly payment.
- Examples include web-based email, CRM, marketing automation and business analytics.

8.4 Comparison Table

Model	What user receives	What user manages	Example
IaaS	Virtual machines, storage, networking	OS, applications, data	Virtual server
PaaS	Development platform, runtime, database, tools	Application and data	Google App Engine, AWS Lambda
SaaS	Complete ready-to-use application	Mostly user account and settings	Web email, online office tools

8.5 Transportation Analogy

Cloud model	Transport analogy	Meaning
IaaS	Rent a vehicle	You get basic resources and control many parts yourself.
PaaS	Use a taxi	The platform is ready; you focus on your work/application.
SaaS	Use public transport	The complete service is ready to use.

8.6 Responsibility Diagram

On-premises: User manages applications, data, runtime, OS, virtualization, networking, storage and servers.

IaaS: Provider manages physical infrastructure; user manages OS, applications and data.

PaaS: Provider manages infrastructure and platform; user mainly manages application and data.

SaaS: Provider manages almost everything; user simply uses the application.

8.7 Common Confusion

- IaaS gives more control but requires more management.
- SaaS is easiest to use but gives the least technical control.
- PaaS is best for developers who want to focus mainly on coding.

Chapter 9: Fog Computing

9.1 Simple Definition

Fog computing is an extension of cloud computing in which processing is done close to IoT devices using nearby edge or fog nodes.

9.2 Why Is It Called Fog?

In nature, fog is closer to the ground than clouds. Similarly, fog computing is closer to end-users and IoT devices than a distant cloud data centre.

9.3 Key Points

- Cisco introduced the term fog computing, also called fogging, in 2014.
- Fog and cloud computing work together.
- Fog nodes are directly connected to physical IoT devices.
- Fog nodes can process large amounts of data locally.
- Only useful or filtered data is sent to the cloud.
- Fog computing reduces delay and internet bandwidth use.

9.4 Cloud vs Fog Data Flow

Cloud computing:
IoT Sensor → Internet → Cloud → Decision / Storage

Fog computing:
IoT Sensor → Nearby Fog Node → Quick local decision
■ Selected data → Cloud storage / analysis

9.5 Comparison: Cloud vs Fog

Point	Cloud Computing	Fog Computing
Location	Remote data centre	Near IoT devices / end-users
Response time	May be higher	Usually lower
Bandwidth	More raw data is sent to cloud	Only filtered/useful data is sent
Internet dependency	Often needed for processing	Local processing can continue without cloud
Processing	Large-scale storage and analysis	Quick local processing and control
Best for	Long-term storage, reports, analytics	Real-time alerts and local control

9.6 Smart Farm Example

1. IoT sensors in a farm measure soil moisture, temperature and other conditions.
2. A microcontroller node collects sensor data.
3. The data is sent to a fog device or local computer.
4. The fog device analyses the data and can control irrigation locally.
5. Important data is sent to the cloud provider.
6. The cloud stores historical data and supports remote monitoring and analysis.

9.7 Applications

- Smart farming and agriculture.
- Smart cities.

- Industrial monitoring.
- Healthcare monitoring.
- Smart homes and buildings.

Chapter 10: IoT Cloud and Blynk

10.1 How IoT Works With Cloud

An IoT system consists of sensors or devices that communicate with the cloud through a network connection. After data reaches the cloud, software processes it and may perform an action, such as sending an alert or automatically adjusting a device.

> **Sensor / Device** → Network connectivity → **Cloud** → Processing → Alert / Device control

10.2 What Is Blynk?

Blynk is a complete IoT cloud software suite used to prototype, deploy and remotely manage connected electronic devices. It is a **PaaS-type** IoT cloud service.

10.3 Features of Blynk

- Connects hardware to the cloud.
- Creates iOS, Android and web applications.
- Shows real-time and historical device data.
- Allows remote control of devices from anywhere.
- Sends notifications and alerts.
- Supports personal projects as well as commercial IoT products.

10.4 Other IoT Cloud Platforms Mentioned

- Fogwing.
- Google Cloud IoT Core.

10.5 What You Need to Start Blynk

Requirement	Examples
Smartphone	Android OS v4.2+ or iOS v9+
IoT board	Arduino, ESP8266/NodeMCU, ESP32, Raspberry Pi, Particle, Nordic nRF51/nRF52
IoT hardware	Sensors and actuators
Network	Wi-Fi, cellular or Ethernet connection

Chapter 11: Blynk Components and Virtual Pins

11.1 Components of Blynk Platform

Component	Simple explanation
Blynk.Console	A web application used to configure and manage devices, data, users, organizations and locations. It also supports remote monitoring and control.
Blynk.Apps	Native iOS and Android mobile apps used to monitor, control and configure devices. They can also automate device work.
Blynk.Edgent	A lightweight embedded library used on supported hardware. Edge + Agent = Edgent.
Blynk.Cloud	The server infrastructure and heart of the Blynk IoT platform. It connects all platform components together.

11.2 Functions of Blynk.Edgent

- Device claiming and Wi-Fi provisioning.
- Bringing a device online and authenticating it with a user.
- Connectivity management using Wi-Fi, cellular or Ethernet.
- Data transfer between device and cloud.
- API support for Blynk.Apps and Blynk.Cloud features.
- Over-the-air firmware updates for selected hardware models.

11.3 What Is a Virtual Pin?

A **virtual pin** is a software channel used to exchange data between the Blynk dashboard and the hardware. It is not a physical Arduino pin.

11.4 Reading From Virtual Pin: BLYNK_WRITE()

BLYNK_WRITE() reads a value sent from the Blynk dashboard to the hardware.

```
BLYNK_WRITE(V1) {
int pinValue = param.asInt();
// use pinValue to control hardware
}
```

Part	Meaning
BLYNK_WRITE(V1)	Runs whenever the value of virtual pin V1 changes.
param.asInt()	Converts the incoming dashboard value into an integer.
pinValue	Variable that stores the received value.

11.5 Writing To Virtual Pin: Blynk.virtualWrite()

Blynk.virtualWrite() sends a value from the hardware board to the Blynk dashboard.

```
Blynk.virtualWrite(V5, 1000);
```

- V5 is the virtual pin connected to a dashboard widget.
- 1000 is the value sent from the board to the dashboard.
- This can be used to display sensor readings on the app.

11.6 Comparison

Function	Direction	Use
BLYNK_WRITE(Vpin)	Dashboard → Hardware	Reads a value from the dashboard.
Blynk.virtualWrite(Vpin, value)	Hardware → Dashboard	Sends a value to the dashboard.

Chapter 12: Blynk Cloud With NodeMCU

12.1 Important Code Parts

```
#define BLYNK_TEMPLATE_ID "Your_Template_ID"
#define BLYNK_DEVICE_NAME "Your_Device_Name"
#define BLYNK_AUTH_TOKEN "Your_Auth_Token"

#define BLYNK_PRINT Serial

#include
#include

char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Your_WiFi_Name";
char pass[] = "Your_WiFi_Password";
```

12.2 Line-by-Line Explanation

Code	Meaning
BLYNK_TEMPLATE_ID	Identifies the Blynk template created in Blynk Cloud.
BLYNK_DEVICE_NAME	Gives a name to the IoT device.
BLYNK_AUTH_TOKEN	Secret token used to authenticate the device with Blynk Cloud.
BLYNK_PRINT Serial	Enables debug messages on the Serial Monitor.
#include	Adds Wi-Fi support for ESP8266.
#include	Adds Blynk cloud support for ESP8266.
auth[]	Stores the authentication token.
ssid[]	Stores the Wi-Fi network name.
pass[]	Stores the Wi-Fi password.

Safety note: Never share your real Blynk authentication token, Wi-Fi name or Wi-Fi password publicly. Treat them like passwords.

12.3 Blynk Event Functions

```
BLYNK_WRITE(V0) {
int value = param.asInt();
Blynk.virtualWrite(V1, value);
}

BLYNK_CONNECTED() {
// Runs when device connects to Blynk Cloud
}
```

12.4 Explanation

- BLYNK_WRITE(V0) runs when virtual pin V0 changes.
- param.asInt() converts the received value into an integer.
- Blynk.virtualWrite(V1, value) sends the same value to virtual pin V1.
- BLYNK_CONNECTED() runs when the device connects successfully to Blynk Cloud.

12.5 setup() and loop()

```
void setup() {
Serial.begin(115200);
Blynk.begin(auth, ssid, pass);
}

void loop() {
Blynk.run();
}
```

Code	Meaning
Serial.begin(115200)	Starts serial communication at 115200 baud for debugging.
Blynk.begin(auth, ssid, pass)	Connects NodeMCU to Wi-Fi and Blynk Cloud.
Blynk.run()	Maintains communication with Blynk Cloud and handles Blynk events.

12.6 Important Note

> Avoid using delay() in the main Blynk loop because it can interrupt Blynk.run() and affect cloud communication.

Chapter 13: Common IoT Application Architecture

13.1 Main Components

Component	Role	Examples
Sensor nodes	Collect data from the environment	Temperature sensor, humidity sensor, GPS
Controller nodes	Collect sensor data and apply logic	Arduino, ESP8266, ESP32
Actuator nodes	Perform physical actions	Relay, alarm, motor, light indicator
Link / network	Connects devices	Wire, LAN, Wi-Fi, Bluetooth
Fog / local computer	Local processing, analysis and control	Local gateway or computer
Internet gateway	Connects local system to internet	Router / gateway
Cloud services	Remote storage, analysis and monitoring	Blynk Cloud, cloud database

13.2 Architecture Diagram

Sensor Nodes → **Controller Node** → **Fog / Local Computer** → **Internet Gateway** → **Cloud Services**
↑ ↓
Actuator Nodes ← Local control / alerts ← Remote monitoring and data analytics

13.3 How It Works Step by Step

1. Sensors and actuators are placed at suitable locations.
2. Sensors and actuators are connected to controller nodes or directly to the cloud.
3. Sensor nodes send measured data to controller nodes or cloud server.
4. The controller node collects data from all sensors.
5. The controller applies business logic according to the requirement.
6. The controller sends data to fog/local computer or directly to cloud.
7. The fog/local computer receives data, analyses it and sends alerts to the local team.
8. It sends filtered data to the cloud.
9. The cloud stores data, performs analysis and allows authorised users to access information.
10. Remote monitoring teams and data analysts use cloud data for decisions.
11. Actuators receive commands and perform actions.

13.4 Role of Fog/Local Computer

- Receives data from controller nodes.
- Executes business logic.
- Performs local analysis.
- Sends only filtered and limited data to cloud.
- Sends alert messages to the local team.
- Needs internet connectivity to communicate with cloud.

13.5 Role of Cloud

- Stores data received from fog/local computer or directly from sensors.
- Executes business logic.
- Performs analysis on stored data.

- Provides information to authorised users.
- Supports remote monitoring and data analytics.

Chapter 14: Quick Revision and Exam Preparation

14.1 Important Definitions

Term	Definition
UART	Universal Asynchronous Receiver-Transmitter; a serial protocol for device-to-device communication.
Baud rate	Communication speed in bits per second.
Serial communication	Sending data one bit at a time.
SoftwareSerial	Arduino library that creates an extra serial port using digital pins.
Cloud computing	Delivery of computing services over the internet.
IaaS	Infrastructure as a Service; provides virtual computing resources.
PaaS	Platform as a Service; provides a development platform.
SaaS	Software as a Service; provides a complete application.
Fog computing	Processing data near IoT devices using edge/fog nodes.
Blynk	IoT cloud platform for connecting, monitoring and controlling devices.
Virtual pin	Software channel for data exchange between Blynk dashboard and hardware.

14.2 Important Syntax

```
Serial.begin(9600);
Serial.available();
Serial.read();
Serial.write(data);
Serial.print("Text");
Serial.println("Text");

#include
SoftwareSerial mySerial(RX_PIN, TX_PIN);

BLYNK_WRITE(V1) {
int value = param.asInt();
}

Blynk.virtualWrite(V5, 1000);
Blynk.begin(auth, ssid, pass);
Blynk.run();
```

14.3 Short Answer Questions

Question	Answer
What is UART?	UART is Universal Asynchronous Receiver-Transmitter, a serial communication protocol used to send data between devices.
What is baud rate?	Baud rate is the speed of serial communication, measured in bits per second.
Which pins are used for UART in Arduino UNO?	Pin 0 is RX and pin 1 is TX.
Difference between print() and println()?	println() adds a new line after output; print() does not.
Difference between read() and peek()?	read() removes the byte from buffer; peek() reads it without removing it.
Why is SoftwareSerial used?	To create additional serial ports using digital pins when hardware UART is busy.
What is cloud computing?	Delivery of computing services such as storage, servers and software over the internet.
What is fog computing?	Processing IoT data close to devices using nearby fog or edge nodes.
What is Blynk?	An IoT cloud platform used to connect, monitor and remotely control devices.
What is a virtual pin?	A software channel used to exchange values between Blynk dashboard and hardware.

14.4 Long Answer Questions

- Explain UART serial communication with its working, pins and advantages.

- Explain the functions of the Arduino Serial class with examples.
- Explain communication between two Arduino boards using UART with code.
- Explain SoftwareSerial, its advantages, limitations and example code.
- Explain cloud computing and its benefits.
- Explain IaaS, PaaS and SaaS with examples and comparison.
- Explain fog computing and compare it with cloud computing.
- Explain the Blynk IoT platform and its components.
- Explain BLYNK_WRITE() and Blynk.virtualWrite() with syntax and examples.
- Explain the common architecture of an IoT application with a diagram.

14.5 MCQs

No.	Question	Options	Correct	Explanation
1	UART stands for:	A) Universal Asynchronous Receiver-Transmitter B) Universal Automatic Remote Transmission C) Unified Asynchronous Radio Transmitter D) Universal Applied Serial Transfer	A	UART means Universal Asynchronous Receiver-Transmitter.
2	In Arduino UNO, TX is available on:	A) Pin 0 B) Pin 1 C) Pin 10 D) Pin 13	B	Pin 1 is TX0 and pin 0 is RX0.
3	Which function starts serial communication?	A) Serial.start() B) Serial.begin() C) Serial.open() D) Serial.connect()	B	Serial.begin(baud_rate) starts serial communication.
4	Which function reads data without removing it from the buffer?	A) Serial.read() B) Serial.write() C) Serial.peek() D) Serial.flush()	C	peek() reads the next byte without removing it.
5	SoftwareSerial is used to:	A) Increase Arduino memory B) Create extra serial ports C) Connect directly to cloud only D) Replace sensors	B	It creates additional serial ports on digital pins.
6	Which cloud model provides a complete application?	A) IaaS B) PaaS C) SaaS D) FaaS	C	SaaS provides a complete ready-to-use application.
7	Which cloud model provides virtual machines and raw infrastructure?	A) SaaS B) PaaS C) IaaS D) BaaS	C	IaaS provides processing, storage and networking resources.
8	Fog computing processes data:	A) Only in the cloud B) Near IoT devices C) Only on mobile phones D) Only in databases	B	Fog computing uses nearby edge/fog nodes.
9	Blynk is mainly which type of cloud service?	A) IaaS B) PaaS C) SaaS D) DaaS	B	The PPT describes Blynk as a PaaS-type cloud service.
10	Which Blynk function sends a value from hardware to dashboard?	A) BLYNK_WRITE() B) Blynk.virtualWrite() C) Serial.read() D) Blynk.connect()	B	virtualWrite() sends a value from board to dashboard.

14.6 Last-Minute Revision Checklist

- UART is asynchronous serial communication.
- TX connects to RX; RX connects to TX.
- Arduino UNO uses pins 0 and 1 for hardware UART.
- Disconnect wires from pins 0 and 1 before uploading code.
- Serial.begin() starts communication.
- Serial.read() removes data from buffer; Serial.peek() does not.
- Serial.println() adds a new line.
- SoftwareSerial creates extra serial ports but is slower.
- Cloud computing provides services over the internet.
- IaaS = infrastructure, PaaS = platform, SaaS = software.
- Fog computing processes data near devices.
- Blynk is a PaaS-type IoT cloud platform.
- BLYNK_WRITE() reads from dashboard; virtualWrite() sends to dashboard.
- Blynk.run() must be called repeatedly in loop().
- IoT architecture: sensors, controller, fog, cloud and actuators.

Final memory line: UART communicates, SoftwareSerial adds ports, Cloud stores and analyses, Fog processes nearby, and Blynk connects IoT devices to dashboards.

End of Complete Study Notes

Revise the tables, diagrams, code explanations and MCQs before the exam.